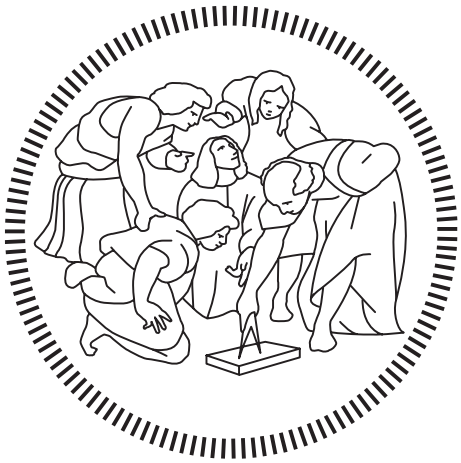
Politecnico di Milano

SCHOOL OF INDUSTRIAL AND INFORMATION ENGINEERING

DIGITAL FACTORY – Prof. Marcello Urgo



Project Work
DFE Assignment
1st Stage

Academic Year 2026 – 2027

Table of Contents

Table of Contents **3**

Chapter 1 Introduction **5**
1.1 Context 5
1.2 Learning Objectives 5

Chapter 2 Assignment Description **7**
2.1 General Scenario 7
2.2 Initial Configuration 8
2.2.1 Source Area 8
2.2.2 Destination Area 8
2.2.3 Obstacle Area 8
2.2.4 Additional Assets 8
2.3 Goal of the Assignment 9
2.4 Provided Resources 10

Chapter 3 Constraints **11**
3.1 Requirements 11
3.2 Physical Consistency 11
3.3 Valid Execution 12
3.4 Convention 12
3.5 Individual Work 13

Chapter 4 Degrees of Freedom **14**
4.1 Factory Layout Design 14
4.2 Assets 15
4.3 Behaviour and Control Strategies 15
4.3.1 Animation-Based Solutions 16
4.3.2 StateChart-Based Solutions 16
4.3.3 Sensor-Actuator Solutions 16
4.3.4 MQTT-Based Solutions 16

Chapter 5 Assessment Criteria **17**
5.1 Functional Correctness 17
5.2 Realism of Dynamics 17
5.3 Safety and Robustness 18
5.4 Quality of Part Placement 18
5.5 Efficiency 18
5.6 Use of Advanced Features 19
5.7 Documentation Quality 19

Chapter 6 Deliverables 20
6.1 Digital Model 20
6.2 Technical Report 20
6.3 Demonstration Video 20

Chapter 7 Documentation 21

Chapter 8 Resources 22
8.1 Project Files 22
8.1.1 Spreadsheet File 22
8.1.2 Scene JSON File 22
8.1.3 Environment JSON File 23
8.1.4 Animation JSON File 23
8.2 Visualization and Testing 23
8.3 Available Asset Libraries 23
8.4 Repository Usage 24

Chapter 9 Tools 25
9.1 VEB.js 25
9.2 OntoGuiWeb 25
9.3 3D Modelling Tools 26
9.3.1 Blender 26
9.4 Text and JSON Editors 26

Chapter 1 Introduction

1.1 Context

The Digital Factory Environment (DFE) project work focuses on the design, implementation, and validation of material handling systems within a virtual factory environment. Students will work on a realistic engineering problem using the tools and modelling concepts provided by the Virtual Learning Factory Toolkit (VLFT). The objective is not simply to create a 3D scene, but to develop a functional transportation system that satisfies operational requirements while respecting physical constraints.

The project exploits the Digital Factory ecosystem, which supports:

- factory asset modelling;
- digital twin instantiation;
- behavioural modelling through StateCharts;
- animation-based simulation;
- physics-enabled execution;
- integration of sensors, actuators, and external control systems.

The reference documentation is available at:

- https://virtualfactory.gitbook.io/vlft
    - Knowledge Base: https://virtualfactory.gitbook.io/vlft/kb
    - VEB.js: https://virtualfactory.gitbook.io/vlft/tools/vebjs

1.2 Learning Objectives

The project work aims to develop practical skills in digital factory engineering through the reuse, customization, and extension of existing virtual factory assets.

Upon completion of the project, students should be able to:

- instantiate and modify factory models;
- design a transportation system satisfying given logistics requirements;
- configure assets and layouts in a virtual factory environment;
- exploit conveyors, robots, sensors, and other factory assets;

- model behaviours using animations and/or StateCharts;
- validate a digital-twin solution using simulation;
- evaluate alternative engineering solutions according to performance, realism, and safety criteria;
- document modelling assumptions, implementation choices, and validation results.

Chapter 2 Assignment Description

2.1 General Scenario

You are provided with a virtual factory environment containing a material source, a destination area, and one or more obstacles. Your task consists of designing and implementing a material handling system able to transport a set of parts from the source location to the destination location.

The project shall be developed within the Digital Factory ecosystem and validated through the VEB.js platform.

The assignment is intentionally formulated as an engineering design problem. Different solutions may satisfy the requirements and students are encouraged to compare alternative layouts, transportation technologies, and control approaches.

Several assignment variants are available:

- DFE_01
- DFE_02
- ...
- DFE_08

Each student will be assigned one specific variant.

The variants share the same general structure and similar expected complexity. The main differences involve:

- placement of the destination box;
- placement of obstacles;
- resulting transportation route.

The transportation distance and overall complexity have been designed to be comparable across all variants.

2.2 Initial Configuration

The provided scene already contains the key elements of the transportation problem.

2.2.1 Source Area

A conveyor (`B1`) contains ten parts that must be transported to the destination area. The conveyor is already configured with a working StateChart controlling forward and backward motions.

2.2.2 Destination Area

A large destination box (`Box`) is placed in the factory. All parts must eventually be placed inside this box.

2.2.3 Obstacle Area

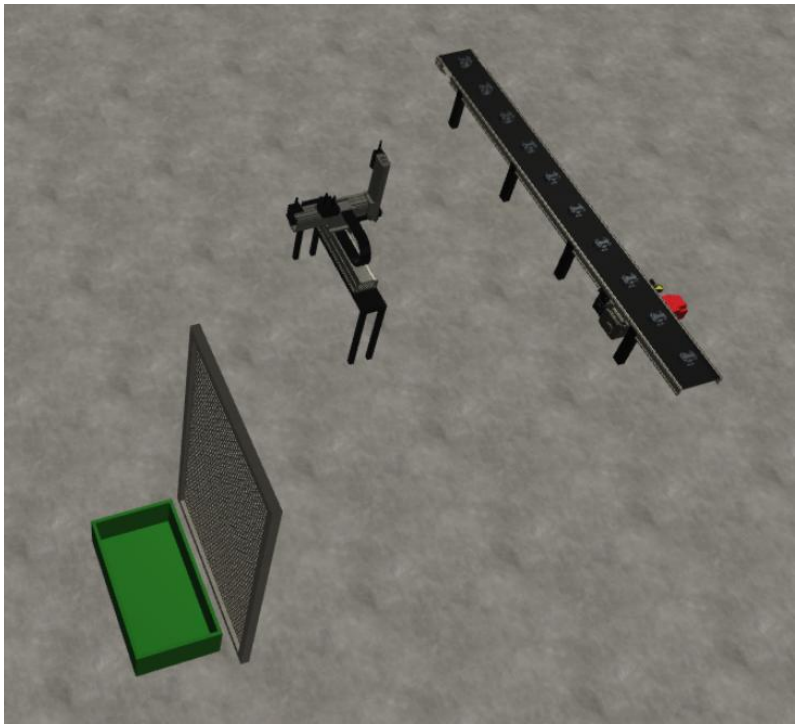
A grid structure (`Grid`) is positioned between the source and the destination.

2.2.4 Additional Assets

The scene also contains a Cartesian robot (`PP`) equipped with:

- Cartesian axes;
- gripper;
- StateChart describing default pick and place motions.

The robot may be reused, customized, or ignored depending on the chosen solution strategy. The use of robotics is not mandatory but may contribute positively to the quality of the solution.



2.3 Goal of the Assignment

The objective is to design, implement, and validate a transportation system capable of:

1. moving all ten parts from the source conveyor to the destination box;
2. avoiding unrealistic behaviours;
3. respecting physical interactions;
4. operating reliably within the virtual factory.

The solution shall be executable in VEB.js and must be reproducible.

The project should be approached as a material handling design problem rather than as a purely modelling exercise.

Students are expected to evaluate questions such as:

- Which transportation assets should be used?
- How should the route be organized?
- Should transportation rely only on conveyors or also on robotic manipulation?
- Can existing assets be improved?
- Would a newly developed asset simplify the solution?

There is no unique correct answer. The quality of the solution will depend on the engineering choices adopted and on the ability to justify them.

2.4 Provided Resources

For the assigned variant `DFE_XY`, the following files are available:

- Spreadsheet model: `DFE_XY.xlsm`
- Scene definition: `DFE_XY.json`
- Environment definition: `DFE_XY_env.json`
- Animation definition: `DFE_XY_anim.json`

The spreadsheet file contains the buttons:

- **Export Scene JSON**
- **Export Animation JSON**

which automatically generate the corresponding JSON files after modifications.

Chapter 3 Constraints

The objective of this section is to define the requirements that every solution must satisfy, independently of the transportation strategy, asset selection, or control approach adopted.

3.1 Requirements

The initial configuration contains a set of assets whose placement shall not be modified.

The following assets must remain in their original position and orientation:

- Source conveyor **B1**
- Destination **Box**
- Obstacle **Grid**

These assets define the transportation problem and therefore cannot be relocated.

The transportation system shall move **all ten parts** initially placed on conveyor B1 into the destination box. The solution shall therefore:

- transport every part;
- ensure that every part reaches the destination area;
- place all parts inside the destination box.

However, it is recommended to reduce the number of parts while testing a candidate solution.

3.2 Physical Consistency

The transportation process shall respect the physical behaviour of the digital twin. Examples of acceptable transportation mechanisms include:

- conveyors;
- robots and manipulators;
- gravity-assisted motions;
- physically simulated interactions;

- sensor-actuator systems.

The following types of solutions are not allowed:

- teleportation of parts;
- instantaneous relocation of parts without a transportation mechanism;
- transportation that ignores collisions or physical interactions.

The provided environment already includes a physics engine (Havok) and gravity configuration that can be exploited during validation.

3.3 Valid Execution

The submitted model shall execute correctly, in particular:

- the scene shall load correctly;
- the transportation sequence shall execute without errors;
- the model shall be reproducible by the instructors.

Students are encouraged to validate the model using VEB.js before submission:

https://virtualfactory.gitbook.io/vlft/tools/vebjs

All assets referenced by the submitted model shall be accessible.

When new assets are introduced, students shall ensure that:

- 3D models are available in an accessible repository;
- file paths are correctly specified;
- custom resources are included with the submission or properly referenced.

The use of local or remote repositories is supported by VEB.js. Relevant documentation can be found at:

- Local Repository
  https://virtualfactory.gitbook.io/vlft/kb/instantiation/datarepo/local
- Remote Repository
  https://virtualfactory.gitbook.io/vlft/kb/instantiation/datarepo/remote

3.4 Convention

The provided resource files use millimetres [mm] as length unit. Therefore:

- positions;
- dimensions;
- placements;
- conveyor lengths;
- robot strokes;
- etc.

must be interpreted in millimetres unless otherwise specified. Students shall maintain consistency with this convention when:

- defining asset dimensions;
- positioning assets;
- designing transportation systems;
- modifying existing assets;
- creating new assets.

3.5 Individual Work

Each assignment variant is intended to be solved independently. Students may consult the documentation and reuse assets available in the provided repositories. However, the submitted transportation system, layout configuration, and technical report shall represent their own work.

Chapter 4 Degrees of Freedom

The project work is intentionally designed as an engineering design challenge rather than a modelling exercise with a predefined solution. While the constraints defined in Section 3 must always be satisfied, students are free to develop different transportation strategies and implementation approaches.

The quality of the solution will depend not only on functional correctness but also on the engineering choices adopted.

4.1 Factory Layout Design

Students may design and implement their own transportation layout provided that the fixed assets identified in Section 3 remain unchanged. There is no prescribed route between the source conveyor and the destination box.

Students are encouraged to reuse assets already available in the Digital Factory ecosystem. Typical examples include:

- conveyors;
- buffers;
- Cartesian robots;
- pick-and-place stations;
- sensors;
- actuators;
- boxes;
- pallets;
- workholding positions;
- etc.

Existing assets can be found in the Digital Factory repositories made available through the project resources: https://difactory.github.io/repository/

4.2 Assets

Existing assets may be modified to improve the transportation system, e.g.:

- changing the dimensions of a conveyor;
- modifying conveyor speed;
- changing robot trajectories;
- modifying StateChart parameters;
- adapting animation parameters;
- redesigning placement positions.

Students should justify why the modifications are useful and how they improve the resulting system.

Students may create new assets when existing assets are insufficient or when alternative solutions can provide significant benefits. New assets may be obtained through:

- modification of existing assets;
- creation of new 3D models;
- creation of new StateCharts;
- creation of new animation behaviours.

There is no mandatory transportation technology. The chosen strategy should be justified from an engineering perspective.

The Cartesian robot provided in the initial model may be:

- reused without modifications;
- repositioned through internal motions;
- enhanced through modifications to its StateChart;
- integrated into larger transportation systems.

The use of the robot is not mandatory. However, effective use of robotic manipulation may contribute positively to the quality of the solution.

4.3 Behaviour and Control Strategies

Different levels of behavioural sophistication are possible.

4.3.1 Animation-Based Solutions

Transportation can be implemented mainly through predefined animations and event scheduling. This approach is generally simpler and may be sufficient to satisfy the project requirements.

4.3.2 StateChart-Based Solutions

Students may exploit StateCharts to model the behaviour of:

- conveyors;
- robots;
- sensors;
- actuators;
- custom transportation devices.

Relevant documentation is available at:

- https://virtualfactory.gitbook.io/vlft/kb/instantiation/statechart

4.3.3 Sensor-Actuator Solutions

Students may exploit sensors and event-driven behaviours to make the transportation system more realistic. Examples include:

- part detection;
- automatic conveyor control;
- conditional transportation logic;
- transfer synchronization.

Such solutions typically reduce the dependence on predefined animation scripts.

4.3.4 MQTT-Based Solutions

Students may implement communication with external controllers through MQTT. Examples include:

- sending events to external applications;
- receiving commands from external controllers;
- coordinating assets through a distributed control architecture.

This approach is optional and may be particularly interesting for students wishing to explore more advanced Digital Twin and Industrial IoT concepts.

Chapter 5 Assessment Criteria

The project work will be evaluated on the basis of the quality of the proposed transportation system, the correctness of the implementation, and the engineering rationale supporting the adopted solution.

The assessment does not depend solely on the ability to complete the transportation task. Students are encouraged to consider realism, robustness, safety, maintainability, and effectiveness when designing their solution.

5.1 Functional Correctness

The first evaluation criterion is the ability of the transportation system to satisfy the project objective.

The solution shall:

- transport all ten parts from the source conveyor to the destination box;
- complete the transportation without manual intervention;
- execute correctly within the VLFT environment;
- produce repeatable results.

5.2 Realism of Dynamics

The transportation process should exhibit realistic behaviour consistent with industrial material handling systems. Examples of positive aspects include:

- realistic conveyor operation;
- realistic robotic movements;
- physically plausible transfers;
- coherent manipulation strategies;
- credible transportation speeds.

Examples of undesirable behaviours include:

- abrupt movements;
- parts passing through assets;

- physically implausible interactions.

5.3 Safety and Robustness

The proposed transportation system should operate safely and reliably.

The evaluation will consider aspects such as:

- collision avoidance;
- stable transportation of parts;
- safe robot trajectories;
- robustness against unexpected interactions;
- proper handling of multiple parts.

5.4 Quality of Part Placement

The final placement of the transported parts inside the destination box will be evaluated. Particular attention will be paid to:

- successful insertion of all parts into the box;
- orderly distribution of parts;
- avoidance of unrealistic piling effects;
- consistency with realistic industrial handling operations.

5.5 Efficiency

Transportation efficiency may contribute positively to the evaluation. Possible indicators include:

- completion time of the transportation process;
- reduction of unnecessary transfers;
- compactness of the layout;
- efficient use of assets;
- reduction of idle movements.

Students should however avoid unrealistic optimizations that compromise safety, robustness, or physical realism.

5.6 Use of Advanced Features

The use of advanced functionalities is optional but may contribute positively to the final evaluation when properly justified. Examples include:

- StateChart-based behaviours;
- sensor-driven solutions;
- event-based control logic;
- MQTT communication with external controllers;
- creation of reusable asset libraries;
- advanced animation strategies;
- customized robotic behaviours.

The mere use of an advanced feature does not automatically result in a better evaluation. The feature should provide a meaningful contribution to the transportation system.

5.7 Documentation Quality

The technical report and demonstration materials should clearly explain:

- the design process;
- the adopted solution;
- the implemented modifications;
- the validation methodology;
- the obtained results.

The evaluation will consider:

- clarity;
- completeness;
- technical accuracy;
- quality of the supporting material.

Students should ensure that another Digital Factory user could understand and reproduce the proposed solution from the provided documentation.

Chapter 6 Deliverables

All submitted material shall be clear, reproducible, and sufficiently documented to allow the professors/tutors to understand, execute, and evaluate the proposed solution.

6.1 Digital Model

Students shall submit the final version of the digital model developed for the assigned project. The submission shall include:

- the updated spreadsheet file;
- the exported Scene JSON file;
- the exported Animation JSON file;
- any additional resources required for execution.

6.2 Technical Report

A concise technical report shall accompany the digital model. Follow the template while avoding unnecessary repetitions.

6.3 Demonstration Video

Students shall provide a short video demonstrating the execution of the transportation system. The video should clearly show:

- the initial configuration;
- the transportation process;
- the final placement of all parts inside the destination box.

The purpose of the video is to facilitate the evaluation process and to provide evidence of successful execution.

Chapter 7 Documentation

Students are encouraged to consult the documentation regularly. The ability to identify, understand, and reuse the available documentation is considered an important component of the project work.

- VLFT Documentation providing an overview of modelling concepts, software tools, available examples, tutorials and use cases.
  https://virtualfactory.gitbook.io/vlft
- The Knowledge Base provides detailed technical documentation related to the modelling and execution of digital factory models
  https://virtualfactory.gitbook.io/vlft/kb
    - Factory Model Instantiation
      https://virtualfactory.gitbook.io/vlft/kb/instantiation
    - Definition of assets
      https://virtualfactory.gitbook.io/vlft/kb/instantiation/assets
    - Spreadsheet-Based Modelling
      https://virtualfactory.gitbook.io/vlft/kb/instantiation/assets/spreadsheet
    - JSON-Based Modelling
      https://virtualfactory.gitbook.io/vlft/kb/instantiation/assets/json
    - StateCharts
      https://virtualfactory.gitbook.io/vlft/kb/instantiation/statechart
    - Animations
      https://virtualfactory.gitbook.io/vlft/kb/instantiation/animations
- Local Repositories
  https://virtualfactory.gitbook.io/vlft/kb/instantiation/datarepo/local
- Remote Repositories
  https://virtualfactory.gitbook.io/vlft/kb/instantiation/datarepo/remote
- VEB.js Documentation
  https://virtualfactory.gitbook.io/vlft/tools/vebjs
- Students are encouraged to analyse existing examples, understand how they are structured, and reuse modelling patterns whenever appropriate:
  https://difactory.github.io/repository/

Chapter 8 Resources

This section describes the files, repositories, and online services that can be used to develop, test, validate, and execute the project work.

Each student is assigned one project variant identified by a code: 01, 02, ..., 08

In the URLs reported below, replace XY with the assigned code.

8.1 Project Files

The following files are provided for each assignment variant.

8.1.1 Spreadsheet File

Primary modelling environment: DFE_XY.xlsm

Available at: https://difactory.github.io/repository/scenes/PW/2026/DFE_XY.xlsm

The provided spreadsheet file includes two buttons that automatically generate the corresponding JSON files after modifications:

- "Export Scene JSON" enerates DFE_XY.json
- "Export Animation JSON" generates DFE_XY_anim.json

Students are encouraged to use these export functions whenever possible instead of manually editing the generated JSON files.

8.1.2 Scene JSON File

Factory model: DFE_XY.json

Available at: https://difactory.github.io/repository/scenes/PW/2026/DFE_XY.json

8.1.3 Environment JSON File

Environment configuration: DFE_XY_env.json

Available at: https://difactory.github.io/repository/scenes/PW/2026/DFE_XY_env.json

The Environment JSON file contains:

- cameras;
- lighting;
- visualization settings;
- WebXR settings;
- physics configuration.

8.1.4 Animation JSON File

Animation and event configuration: DFE_XY_anim.json

Available at: https://difactory.github.io/repository/scenes/PW/2026/DFE_XY_anim.json

The Animation JSON file defines:

- event scheduling;
- animations;
- triggers;
- execution sequences.

8.2 Visualization and Testing

The input project can be visualized and tested using VEB.js at:

https://digital-factory.io/vebjs/?inputscene=https://difactory.github.io/repository/scenes/PW/2026/DFE_XY&repoMod3d=https://difactory.github.io/repository/models/PickPlaceCell/

By replacing **XY** with the assignment code, students can directly load and execute their scenario in VEB.js.

8.3 Available Asset Libraries

A repository of reusable 3D assets is available at:
https://github.com/difactory/repository/tree/main/models

Particularly relevant assets can be found in the PickPlaceCell Asset Library:
https://github.com/difactory/repository/tree/main/models/PickPlaceCell

---

8.4 Repository Usage

Project resources may be stored either:

- On a Remote Repository, e.g. GitHub repositories, GitHub Pages repositories, or other HTTPS-accessible repositories.
- On a Local Repository for development and testing activities.

Chapter 9 Tools

The choice of tools is left to the student. However, the tools listed below are strongly recommended.

9.1 VEB.js

VEB.js is the primary environment for development, validation, debugging, and demonstration of the transportation system: https://virtualfactory.gitbook.io/vlft/tools/vebjs

VEB.js can be used to:

- load and visualize factory scenes;
- execute animations;
- execute StateCharts;
- validate transportation behaviours;
- inspect physical interactions;
- validate custom assets;
- test WebXR interactions.

9.2 OntoGuiWeb

OntoGuiWeb can be used to import and export JSON files and to inspect the semantic structure of factory models. Students may use OntoGuiWeb to:

- inspect assets;
- modify asset properties;
- edit StateCharts;
- analyse scene structures;
- convert and manipulate model definitions.

OntoGuiWeb is particularly useful when working with more complex assets or when a graphical representation of the factory model is preferred over direct editing of a spreadsheet or JSON files.

9.3 3D Modelling Tools

3D modelling software may be used when:

- modifying existing assets;
- creating new assets;
- generating alternative geometric representations.

9.3.1 Blender

The recommended tool is Blender: https://www.blender.org/

Blender supports:

- import of glTF models;
- export of glTF models;
- export of GLB files;
- mesh editing;
- assembly modelling;
- animation authoring.

9.4 Text and JSON Editors

Although the spreadsheet file already generates JSON files automatically, it is often useful to inspect the generated files directly. The use of an advanced text editor is therefore recommended. Examples include:

- Visual Studio Code: https://code.visualstudio.com/
- Notepad++: https://notepad-plus-plus.org/